

Foundations of Intrusion Detection and Log Analysis

Chapter 1: Foundations of Log Analysis and Historical Context

1.1. Historical Background and Evolution of Linux Security

Since the early use of Unix-like operating systems in critical business settings, the methodology for detecting malicious activity has undergone significant change. Early approaches to security monitoring in the 1990s and early 2000s primarily relied on administrators manually reviewing system logs in text form. This model based on focused on reactive measures, assuming that if a perimeter firewall was strong, internal systems were likely safe.

Security concentrated on external network threats, often overlooking the subtle, traceable signs of credential misuse or privilege escalation that are now recognized as critical threats. The rise of automation tools and high-volume data streams necessitated a shift away from manual review toward programmatic analysis, paving the way for our project's approach: using Regular Expressions (regex) and time-series analysis to perform automated Host-based Intrusion Detection HIDS. This methodological evolution is what allows our system to quickly identify high-confidence security incidents by quantifying patterns of failures that were previously lost in the sheer volume of unstructured log data

As internet-facing applications, cloud services, and large distributed systems grew, attackers began exploiting insecure configurations, weak passwords, needlessly open ports, and lax access controls. This shift necessitated a structured methodology for lowering operating system vulnerabilities, resulting in the emergence of system hardening. This methodical approach involves several key steps: minimizing the attack surface, enforcing strict authentication rules, disabling unnecessary services, and applying strong password and access policies. The transition to modern Linux environments has fundamentally redefined how authentication and privilege escalation are handled, moving beyond simple one-time checks to an ongoing, context-aware process.

1.2. The Emergence of Host-based Intrusion Detection (HIDS)

The necessity for **Host-based Intrusion Detection Systems (HIDS)** arose because external firewalls (perimeter defenses) cannot detect internal threats or actions taken by an attacker who has successfully gained an initial foothold. Our project focused on HIDS, which utilizes **Logging and Audit Policies** to provide visibility into system activity, enabling the detection of suspicious or unauthorized behavior.

- **Early Detection:** Initial detection relied on manual review of system logs.
- **Modern HIDS:** Modern HIDS automate this process, using tools like **Regular Expressions (regex)** and scripting to transform unstructured log data into actionable security intelligence.

Chapter 2: Security Models and Attack Vectors in the Project

2.1. Authentication and Privilege Governance

In the past, Linux systems primarily relied on the **root account**, offering complete system control. Any compromise of the root password led to total system takeover, posing significant risks. To mitigate this, the concept of "**least privilege**" was adopted, restricting user access to only what is required.

Tools like **sudo** (used in our Privilege Escalation simulation) and **PAM (Pluggable Authentication Modules)** fundamentally redefined how authentication and privilege escalation are handled. In our project, the goal was to detect the failure of this governance model:

- **Brute Force Detection:** The project simulates the failure of the initial password check.
- **Privilege Escalation Detection:** The project targets the failure of the secondary, post-login check (the **sudo** password prompt). This change ensures that even if a user gains access, their actions remain restricted and traceable, significantly lowering the possibility of unauthorized system modification.

2.2. Network Security and Governance

Network hardening became crucial as Linux systems hosted more internet-facing applications. Modern security governance emphasizes "**default deny**" principles, permitting communication only when specifically allowed, a contrast to the old "default allow" models that exposed superfluous services.

- **Project Context:** Our project utilized **UFW (Uncomplicated Firewall)**, which enforces Network Security Controls by restricting open ports and network exposure. While we explicitly **allowed Port 22 (SSH)** to permit the application-layer attack, the **Port Scanning** simulation confirmed that the UFW was actively enforcing the robustly defended perimeter.
- **Log Differentiation:** The project demonstrated the difference between **Firewall Enforcement** (network security logs) and **Service Failure Detection** (application security logs in `auth.log`).

2.3. Log Analysis as an Audit and Compliance Tool

Linux systems include kernel activity monitors and auditing frameworks like `auditd`. These tools guarantee **traceability** and enable administrators to promptly identify anomalies. Effective auditing allows organizations to reconstruct security incidents and find setup flaws. Our rule-based HIDS directly feeds into this process, generating the **alerts** necessary to uphold adherence to security norms.

Chapter 3: Project Environment and Methodology Deep Dive

3.1. Virtual Lab Environment Specification

The project required a strict, controlled environment to ensure accurate logging and isolated testing.

- **Victim VM (Ubuntu Server):** Chosen for its stability and widespread use in production environments. Target service: **OpenSSH Server**.
- **Attacker VM (Kali Linux):** Chosen for its pre-installed suite of security auditing tools, including ncrack and nmap.
- **Networking Protocol: Bridged Adapter** mode was chosen. This ensured that the Attacker and Victim VMs were on the same logical network segment as the Host PC, crucial for the scp log transfer function.

3.2. Role of Shell Scripting and Python in Automation

While our primary analysis used Python, the system hardening concepts themselves rely heavily on shell scripting.

- **Shell Scripting (Bash):** Historically, system administrators used shell scripts to enforce security policies and automate configuration updates across numerous machines. In our project, commands like `sudo ufw enable` and `sudo adduser` represent security configurations typically managed by shell scripts for consistency and accuracy across systems. Shell scripts are still crucial for preserving stable and secure server environments.
- **Python (Pandas/Regex):** Python was used to encapsulate the complex logic of **data analysis** and **detection**. Tools like Pandas handle the high-volume data processing required, while regex (Regular Expressions) performs the precise pattern matching that is impractical to achieve with standard shell scripting.

3.3. Evaluation Metrics: The Trade-Off Analysis

The core contribution of the project was the quantitative analysis, which requires a deep understanding of the metrics used:

1. **True Positive Rate (TPR) / Recall:** Measures the fraction of actual attacks that were correctly identified as alerts. Maximizing TPR is ideal for security.
2. **False Positive Rate (FPR):** Measures the fraction of normal events that were mistakenly flagged as attacks. **Minimizing FPR** is essential for maintaining operational stability and preventing administrator **alert fatigue**.

Our project directly evaluated the tension between these two goals by comparing **Threshold 1 (5/10 min)** against **Threshold 2 (10/10 min)**. The measurable difference in their FPR dictated the final, practical security recommendation, confirming that effective system hardening requires a nuanced understanding of risk versus operational cost.